Nama: Addien Zakia Hidayat
NRP: 5002231063

Perangkat Lunak Matematika

Tugas 5

Aturan Pengerjaan:

- Tuliskan identitas Anda dengan urutan: Nama, NRP.
- Telusuri program yang Anda buat dan jelaskan output yang dihasilkan pada program yang dibuat.
- Kirimkan file jawaban Anda dalam ekstensi .pdf dengan format nama "Tugas 5-PLM-Nama" paling lambat sebelum kelas dimulai. Keterlambatan pengumpulan jawaban akan berakibat pada penalti nilai. Pengumpulan file jawaban yang terlambat lebih dari satu hari akan dianggap tidak mengerjakan.

In [4]:
```
#Tuliskan library yang Anda perlukan disini
import math
import numpy as np
import matplotlib.pyplot as plt
from sympy import *
from scipy.integrate import quad
import numpy as np
import matplotlib.pyplot as plt
import matplotlib.patches as patches
```

Nomor 1

Gambarkan fungsi $f(x) = e^{-x^2}$ dengan ketentuan sebagai berikut.

- Buat garis putus-putus yang menunjukkan asimtot datar dan asimtot tegak (jika ada).
- Jelaskan kapan fungsi naik dan turun, serta kapan cekung ke atas dan cekung ke bawah terjadi.

$$f'(x) = -2x \cdot e^{-x^2}$$

$$f''(x) = e^{-x^2}(4x^2 - 2)$$

- Titik maks di (0, 1), asimtot datar y = 0
- Titik infleksi di x = ±1/√2, naik/turun dianalisis dari f'(x)

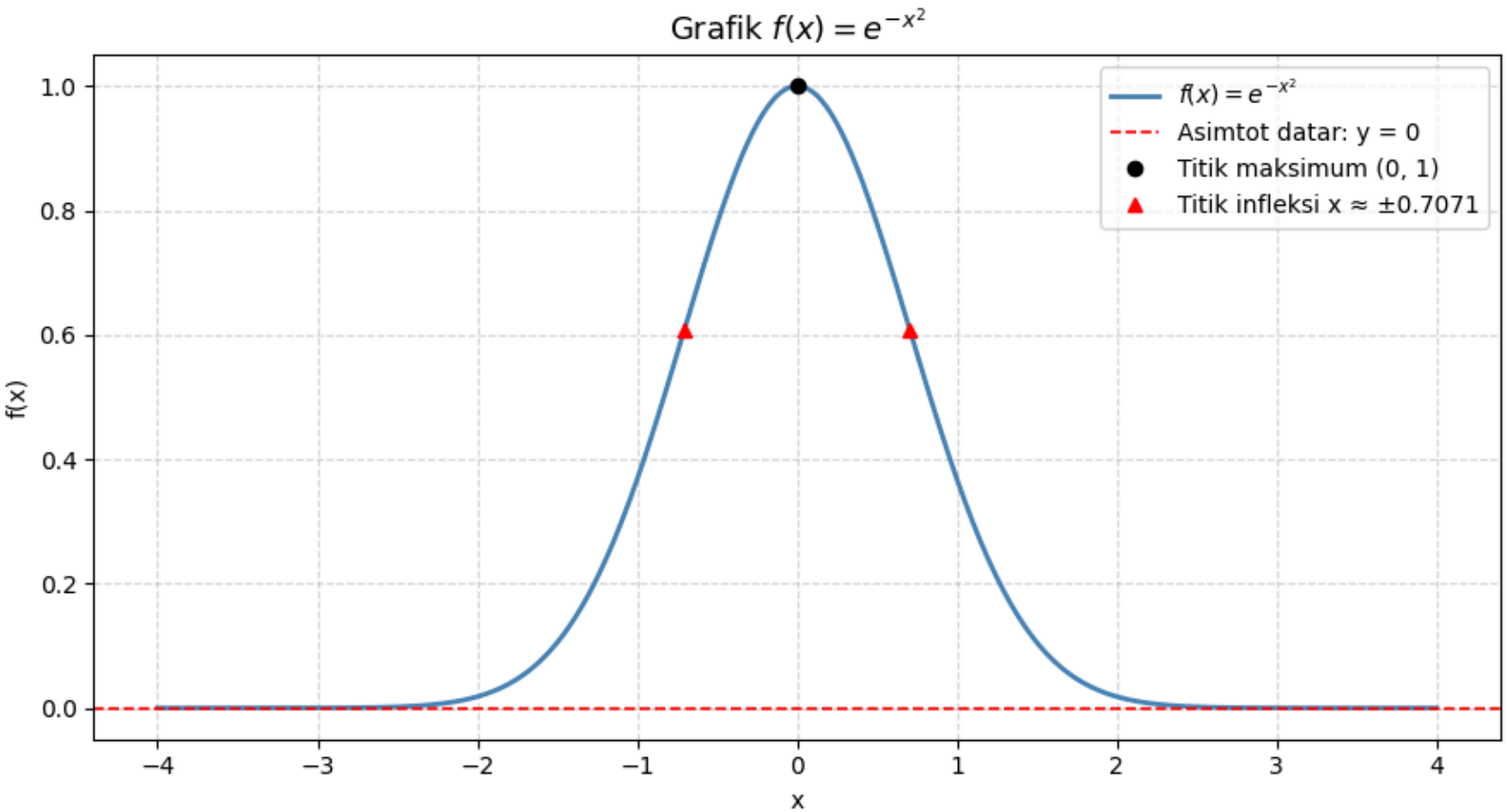
In [5]:
```
## Tuliskan jawaban Anda disini
def nomor_1():
    x_sym = symbols('x')
    fx = exp(-x_sym**2)

    df  = diff(fx, x_sym)      # f'(x)
    ddf = diff(df, x_sym)      # f''(x)

    # Cari titik kritis (f'(x) = 0)
    kritis = solve(df, x_sym)

    # Cari titik infleksi (f''(x) = 0)
    infleksi = solve(ddf, x_sym)

    print("NOMOR 1 - f(x) = e^(-x^2)\n")
    print(f"f'(x)  = {df}")
    print(f"f''(x) = {ddf}")
    print(f"Titik kritis   : x = {kritis}")
    print(f"Titik infleksi : x = {[round(float(v), 4) for v in infleksi]}")

    # Analisis naik/turun
    print("\nAnalisis Naik/Turun:")
    print("  f'(x) = -2x * e^(-x^2)")
    print("  f'(x) > 0  <=>  x < 0  => f naik pada (-inf, 0)")
    print("  f'(x) < 0  <=>  x > 0  => f turun pada (0, +inf)")

    # Analisis cekung
    print("\nAnalisis Cekung:")
    infleksi_val = [float(v) for v in infleksi]
    infleksi_val.sort()
    print(f"  Titik infleksi di x = {[round(v, 4) for v in infleksi_val]}")
    print("  f cekung ke atas  pada (x < -1/sqrt(2)) dan (x > 1/sqrt(2))")
    print("  f cekung ke bawah pada (-1/sqrt(2) < x < 1/sqrt(2))")
```

```python
    # Plot
    x_vals = np.linspace(-4, 4, 500)
    y_vals = np.exp(-x_vals**2)

    fig, ax = plt.subplots(figsize=(9, 5))

    ax.plot(x_vals, y_vals, color='steelblue', linewidth=2, label=r'$f(x)=e^{-x^2}$')
    ax.axhline(y=0, color='red', linestyle='--', linewidth=1.2, label='Asimtot datar: y = 0')
    ax.plot(0, 1, 'ko', markersize=6, label='Titik maksimum (0, 1)')

    for xi in infleksi_val:
        yi = float(fx.subs(x_sym, xi))
        ax.plot(xi, yi, 'r^', markersize=6)

    ax.plot([], [], 'r^', markersize=6, label=f'Titik infleksi x ≈ ±{round(abs(infleksi_val[0]), 4)}')

    ax.set_title(r'Grafik $f(x) = e^{-x^2}$', fontsize=13)
    ax.set_xlabel('x')
    ax.set_ylabel('f(x)')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.show()
```

In [6]:

```python
nomor_1()
```

```
NOMOR 1 - f(x) = e^(-x^2)

f'(x)  = -2*x*exp(-x**2)
f''(x) = 4*x**2*exp(-x**2) - 2*exp(-x**2)
Titik kritis   : x = [0]
Titik infleksi : x = [-0.7071, 0.7071]

Analisis Naik/Turun:
  f'(x) = -2x * e^(-x^2)
  f'(x) > 0  <=>  x < 0  => f naik pada (-inf, 0)
  f'(x) < 0  <=>  x > 0  => f turun pada (0, +inf)

Analisis Cekung:
  Titik infleksi di x = [-0.7071, 0.7071]
  f cekung ke atas  pada (x < -1/sqrt(2)) dan (x > 1/sqrt(2))
  f cekung ke bawah pada (-1/sqrt(2) < x < 1/sqrt(2))
```



Nomor 2

Dapatkan luas daerah A yang terbentuk dari kurva $y = x + 4$ dan $y = x^2 - 2$. Buatlah ilustrasinya.

$$A = \int_{-2}^{3} \left[(x+4) - (x^2 - 2)\right] dx = \frac{125}{6} \approx 20.83$$

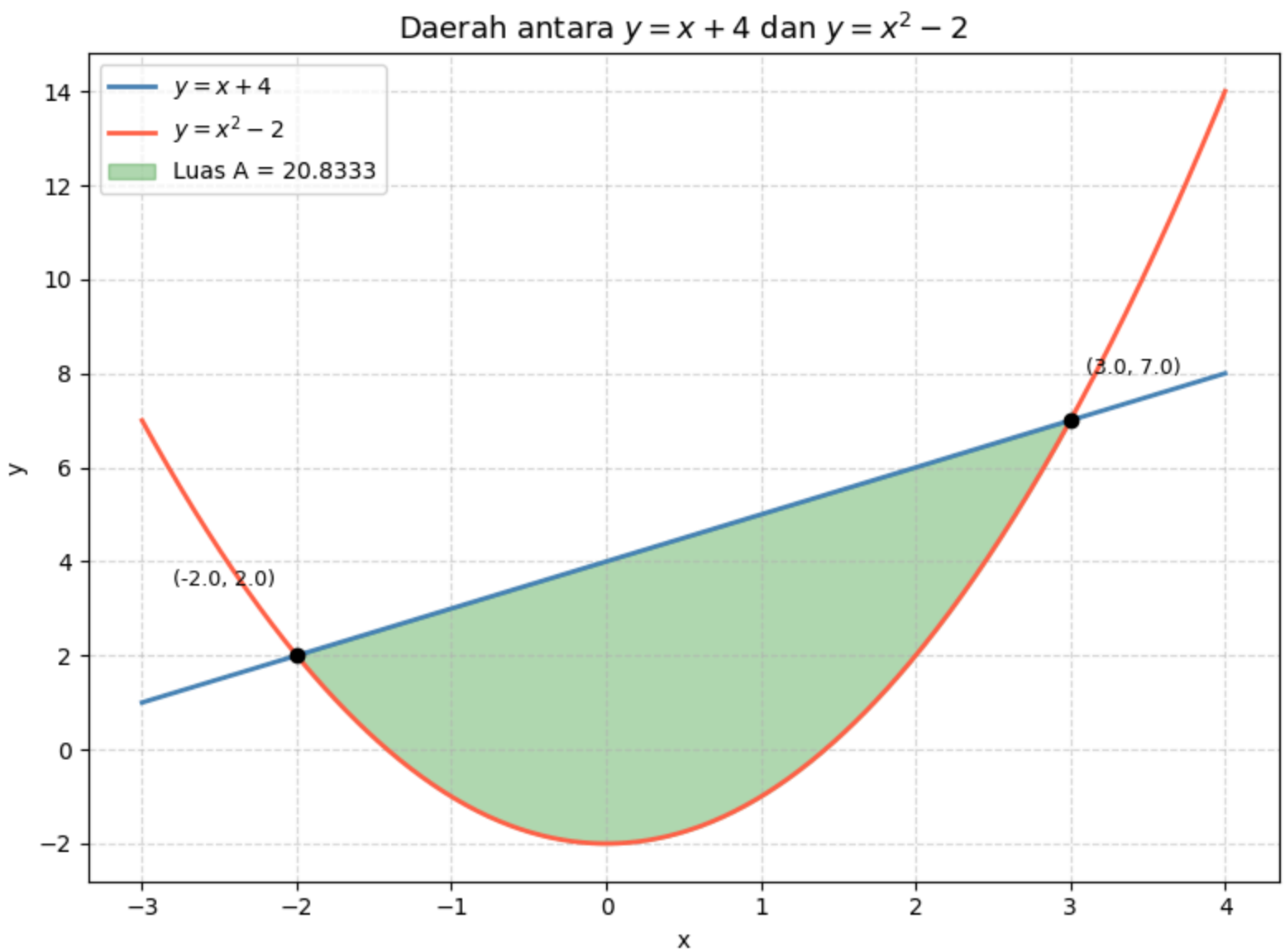
- Titik potong di x = -2 dan x = 3
- y = x+4 selalu di atas y = x^2-2 pada interval [-2, 3]

In [7]:
```
## Tuliskan jawaban Anda disini
def nomor_2():
    x_sym = symbols('x')
    f1 = x_sym + 4
    f2 = x_sym**2 - 2

    # Cari titik potong
    intersections = solve(f1 - f2, x_sym)
    a, b = sorted([float(v) for v in intersections])

    print("NOMOR 2 - Luas Daerah antara y=x+4 dan y=x^2-2\n")
    print(f"Titik potong: x = {a}, x = {b}")

    # Hitung luas secara simbolik
    integrand = f1 - f2
    luas = integrate(integrand, (x_sym, a, b))
    print(f"Luas A = integral dari {a} ke {b} dari (x+4 - (x^2-2)) dx = {luas}")

    # Plot
    x_vals = np.linspace(a - 1, b + 1, 500)
    y1 = x_vals + 4
    y2 = x_vals**2 - 2

    x_fill = np.linspace(a, b, 500)
    y1_fill = x_fill + 4
    y2_fill = x_fill**2 - 2

    fig, ax = plt.subplots(figsize=(8, 6))

    ax.plot(x_vals, y1, color='steelblue', linewidth=2, label=r'$y = x + 4$')
    ax.plot(x_vals, y2, color='tomato', linewidth=2, label=r'$y = x^2 - 2$')
    ax.fill_between(x_fill, y2_fill, y1_fill, alpha=0.3, color='green', label=f'Luas A = {float(luas):.4f}')
```

```
    ax.plot([a, b], [a + 4, b + 4], 'ko', markersize=6)
    ax.annotate(f'({a:.1f}, {a+4:.1f})', xy=(a, a + 4), xytext=(a - 0.8, a + 5.5), fontsize=9)
    ax.annotate(f'({b:.1f}, {b+4:.1f})', xy=(b, b + 4), xytext=(b + 0.1, b + 5.0), fontsize=9)

    ax.set_title('Daerah antara $y=x+4$ dan $y=x^2-2$', fontsize=13)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.5)
    plt.tight_layout()
    plt.show()
```

In [8]:
```
nomor_2()
```

```
NOMOR 2 - Luas Daerah antara y=x+4 dan y=x^2-2

Titik potong: x = -2.0, x = 3.0
Luas A = integral dari -2.0 ke 3.0 dari (x+4 - (x^2-2)) dx = 20.8333333333333
```



Nomor 3

Gunakan metode Cakram untuk menghitung volume benda putar dari daerah R yang dibatasi oleh kurva $y = 3 + 2x - x^2$, diputar terhadap

a. Sumbu-x,

b. Sumbu-y,

c. Garis $y = -1$,

d. Garis $x = 4$.

Buatlah ilustrasi dari masing-masing kasus.

$$V = \pi \int_a^b [R(x)]^2 \, dx \quad (\text{sumbu-}x)$$

$$V = \pi \int_c^d \left[R^2 - r^2\right] dy \quad (\text{metode cincin})$$

- Putar sumbu-x: r = f(x), integrasi arah dx
- Putar sumbu-y atau garis horizontal: balik fungsi → x = g(y)
- Putar y = -1: jari-jari cakram = f(x) + 1

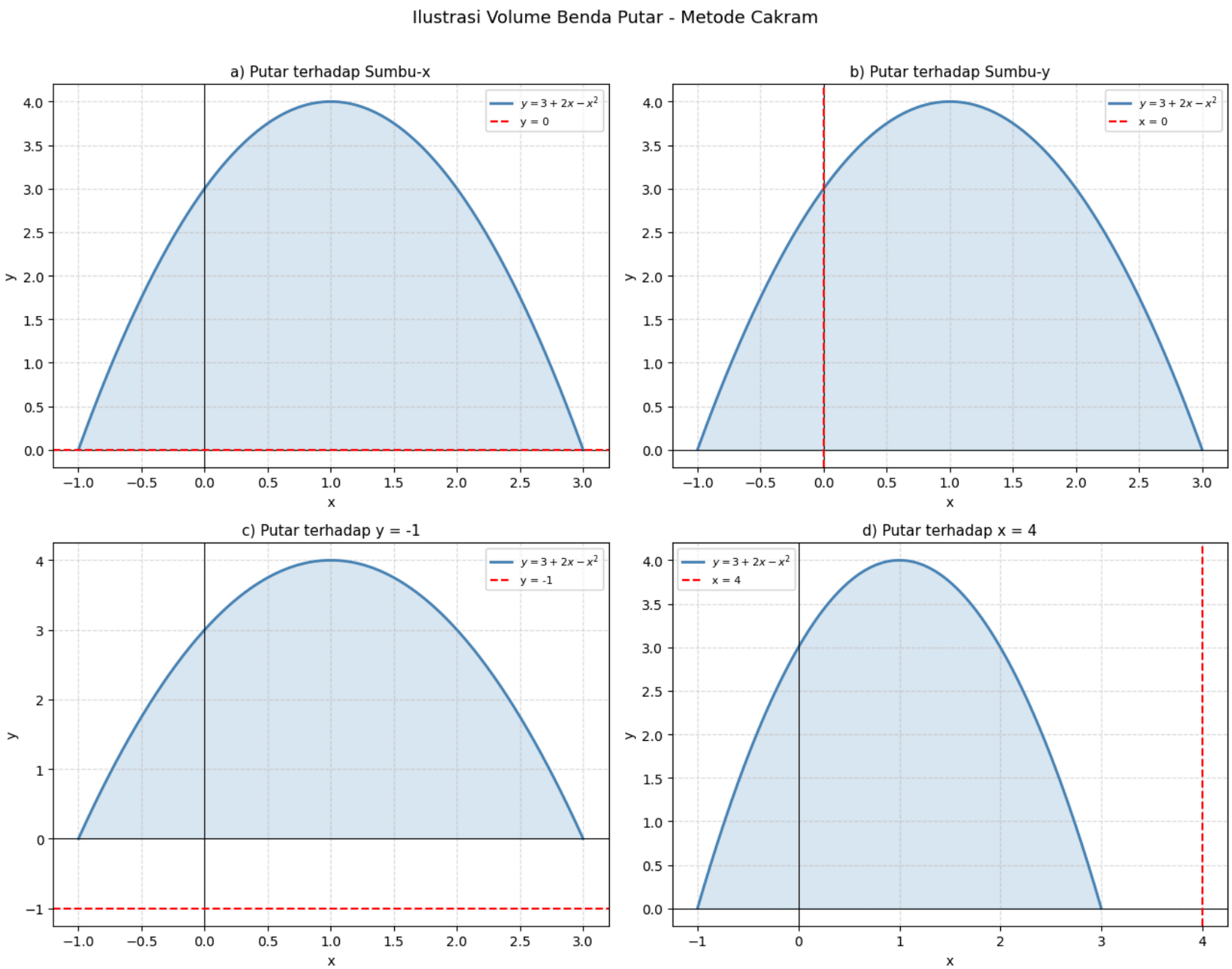
In [9]:
```
## Tuliskan jawaban Anda disini
def nomor_3():
    x_sym, y_sym = symbols('x y')
    f = 3 + 2*x_sym - x_sym**2

    # Cari batas (f(x) >= 0)
    roots = solve(f, x_sym)
    x_a, x_b = sorted([float(v) for v in roots])

    print("NOMOR 3 - Volume Benda Putar y = 3+2x-x^2\n")
    print(f"Kurva memotong sumbu-x di x = {x_a} dan x = {x_b}")

    # Cari batas y (f(y) untuk sumbu-y)
    # Ungkapkan x dalam y: y = 3+2x-x^2 => x = 1 ± sqrt(4-y)
    y_max = float(f.subs(x_sym, 1))  # titik puncak di x=1
    print(f"Titik puncak parabola: ({1}, {y_max})")
```

```python
# --- a) Putar terhadap sumbu-x ---
V_a = integrate(pi * f**2, (x_sym, x_a, x_b))
print(f"\na) Volume putar sumbu-x = {V_a} ≈ {float(V_a):.6f}")

# --- b) Putar terhadap sumbu-y ---
# Metode cakram arah y: x dari kanan - kiri
# x_kanan = 1 + sqrt(4 - y), x_kiri = 1 - sqrt(4 - y)
x_kanan = 1 + sqrt(4 - y_sym)
x_kiri  = 1 - sqrt(4 - y_sym)
V_b = integrate(pi * (x_kanan**2 - x_kiri**2), (y_sym, 0, y_max))
print(f"b) Volume putar sumbu-y = {V_b} ≈ {float(V_b):.6f}")

# --- c) Putar terhadap garis y = -1 ---
V_c = integrate(pi * ((f + 1)**2 - (0 + 1)**2), (x_sym, x_a, x_b))
print(f"c) Volume putar y = -1  = {V_c} ≈ {float(V_c):.6f}")

# --- d) Putar terhadap garis x = 4 ---
# Metode cakram: R(y) = 4 - x_kiri, r(y) = 4 - x_kanan
R_y = 4 - x_kiri
r_y = 4 - x_kanan
V_d = integrate(pi * (R_y**2 - r_y**2), (y_sym, 0, y_max))
print(f"d) Volume putar x = 4   = {V_d} ≈ {float(V_d):.6f}")

# --- Ilustrasi 4 subplot ---
x_vals = np.linspace(x_a, x_b, 300)
y_vals = 3 + 2*x_vals - x_vals**2

fig, axes = plt.subplots(2, 2, figsize=(13, 10))
titles = [
    'a) Putar terhadap Sumbu-x',
    'b) Putar terhadap Sumbu-y',
    'c) Putar terhadap y = -1',
    'd) Putar terhadap x = 4',
]
hlines = [0, None, -1, None]
vlines = [None, 0, None, 4]

for idx, ax in enumerate(axes.flatten()):
    ax.plot(x_vals, y_vals, color='steelblue', linewidth=2, label=r'$y=3+2x-x^2$')
    ax.fill_between(x_vals, 0, y_vals, alpha=0.2, color='steelblue')
    ax.axhline(0, color='black', linewidth=0.8)
    ax.axvline(0, color='black', linewidth=0.8)
```

```python
        if hlines[idx] is not None:
            ax.axhline(hlines[idx], color='red', linestyle='--', linewidth=1.5,
                       label=f'y = {hlines[idx]}')
        if vlines[idx] is not None:
            ax.axvline(vlines[idx], color='red', linestyle='--', linewidth=1.5,
                       label=f'x = {vlines[idx]}')

        ax.set_title(titles[idx], fontsize=11)
        ax.set_xlabel('x')
        ax.set_ylabel('y')
        ax.legend(fontsize=8)
        ax.grid(True, linestyle='--', alpha=0.4)

    plt.suptitle('Ilustrasi Volume Benda Putar - Metode Cakram', fontsize=13, y=1.01)
    plt.tight_layout()
    plt.show()
```

In [10]:
```python
nomor_3()
```

```
NOMOR 3 - Volume Benda Putar y = 3+2x-x^2

Kurva memotong sumbu-x di x = -1.0 dan x = 3.0
Titik puncak parabola: (1, 4.0)

a) Volume putar sumbu-x = 34.1333333333333*pi ≈ 107.233029
b) Volume putar sumbu-y = 64*pi/3 ≈ 67.020643
c) Volume putar y = -1  = 55.4666666666667*pi ≈ 174.253673
d) Volume putar x = 4   = 64*pi ≈ 201.061930
```


Ilustrasi Volume Benda Putar - Metode Cakram
a) Putar terhadap Sumbu-x
$y = 3 + 2x - x^2$
y = 0
b) Putar terhadap Sumbu-y
$y = 3 + 2x - x^2$
x = 0
c) Putar terhadap y = -1
$y = 3 + 2x - x^2$
y = -1
d) Putar terhadap x = 4
$y = 3 + 2x - x^2$
x = 4
x
y

Nomor 4

Buatlah gambar yang mengilustrasikan Metode Newton-Raphson untuk mencari nilai optimum dari suatu fungsi. Buatlah untuk 2 kasus, yaitu 1 kasus yang konvergen, dan 1 kasus yang tidak konvergen.

$$x_{n+1} = x_n - \frac{f'(x_n)}{f''(x_n)}$$

- Konvergen jika $f''(x_0) \neq 0$ dan x_0 cukup dekat ke titik kritis
- Tidak konvergen jika x_0 berada di dekat titik sadel ($f'' \approx 0$)

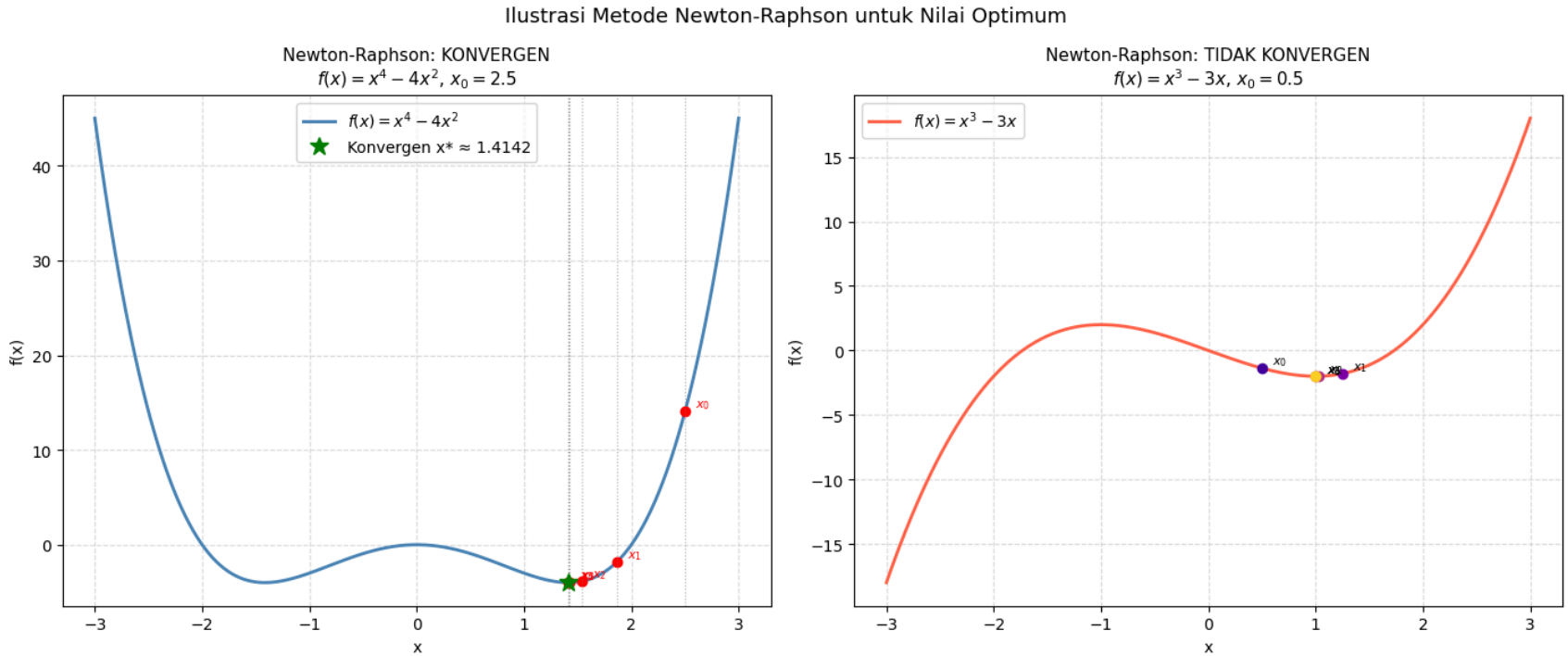
In [11]:
```
## Tuliskan jawaban Anda disini
def nomor_4():
    print("NOMOR 4 - Newton-Raphson Optimum")

    x_sym = symbols('x')

    def iterasi_nr(fx_expr, x0, max_iter=20):
        df  = diff(fx_expr, x_sym)
        ddf = diff(df, x_sym)

        history = [float(x0)]
        x_n = x0

        for i in range(max_iter):
            df_val  = float(df.subs(x_sym, x_n))
            ddf_val = float(ddf.subs(x_sym, x_n))

            if abs(ddf_val) < 1e-12:
                print(f"  f''(x) ≈ 0, iterasi berhenti di i={i}")
                break

            x_np1 = x_n - df_val / ddf_val
            history.append(float(x_np1))

            if abs(x_np1 - x_n) < 1e-6:
```

```
                print(f"  Konvergen di x = {float(x_np1):.6f} setelah {i+1} iterasi")
                break

            x_n = x_np1
        else:
            print(f"  Tidak konvergen setelah {max_iter} iterasi")

        return history

# Kasus 1: Konvergen - f(x) = x^4 - 4x^2, x0 = 2.5
fx_konvergen = x_sym**4 - 4*x_sym**2
x0_konvergen = Float(2.5)
print("\nKasus Konvergen: f(x) = x^4 - 4x^2, x0 = 2.5")
hist_conv = iterasi_nr(fx_konvergen, x0_konvergen)

# Kasus 2: Tidak Konvergen - f(x) = x^3 - 3x, x0 = 0.5 (dekat sadel)
fx_diverge = x_sym**3 - 3*x_sym
x0_diverge = Float(0.5)
print("\nKasus Tidak Konvergen: f(x) = x^3 - 3x, x0 = 0.5")
hist_div = iterasi_nr(fx_diverge, x0_diverge, max_iter=15)

fig, axes = plt.subplots(1, 2, figsize=(14, 6))

# --- Plot kasus konvergen ---
ax = axes[0]
x_range = np.linspace(-3, 3, 500)
y_range = x_range**4 - 4*x_range**2
ax.plot(x_range, y_range, color='steelblue', linewidth=2, label=r'$f(x)=x^4-4x^2$')

df_conv = diff(fx_konvergen, x_sym)
for i, xi in enumerate(hist_conv[:-1]):
    yi     = float(fx_konvergen.subs(x_sym, xi))
    dfi    = float(df_conv.subs(x_sym, xi))
    x_next = hist_conv[i + 1]

    ax.plot(xi, yi, 'ro', markersize=6)
    ax.annotate(f'$x_{i}$', xy=(xi, yi), xytext=(xi + 0.1, yi + 0.5), fontsize=8, color='red')
    ax.axvline(xi, color='gray', linestyle=':', linewidth=0.8, alpha=0.6)

ax.plot(hist_conv[-1], float(fx_konvergen.subs(x_sym, hist_conv[-1])), 'g*',
        markersize=12, label=f'Konvergen x* ≈ {hist_conv[-1]:.4f}')
ax.set_title('Newton-Raphson: KONVERGEN\n$f(x)=x^4-4x^2$, $x_0=2.5$', fontsize=11)
```

```python
    ax.set_xlabel('x')
    ax.set_ylabel('f(x)')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.4)

    # --- Plot kasus tidak konvergen ---
    ax = axes[1]
    x_range2 = np.linspace(-3, 3, 500)
    y_range2 = x_range2**3 - 3*x_range2
    ax.plot(x_range2, y_range2, color='tomato', linewidth=2, label=r'$f(x)=x^3-3x$')

    df_div = diff(fx_diverge, x_sym)
    colors_div = plt.cm.plasma(np.linspace(0.1, 0.9, len(hist_div)))
    for i, xi in enumerate(hist_div):
        try:
            yi = float(fx_diverge.subs(x_sym, xi))
            ax.plot(xi, yi, 'o', color=colors_div[i], markersize=6)
            ax.annotate(f'$x_{i}$', xy=(xi, yi), xytext=(xi + 0.1, yi + 0.3), fontsize=8)
        except Exception:
            break

    ax.set_title('Newton-Raphson: TIDAK KONVERGEN\n$f(x)=x^3-3x$, $x_0=0.5$', fontsize=11)
    ax.set_xlabel('x')
    ax.set_ylabel('f(x)')
    ax.legend()
    ax.grid(True, linestyle='--', alpha=0.4)

    plt.suptitle('Ilustrasi Metode Newton-Raphson untuk Nilai Optimum', fontsize=13)
    plt.tight_layout()
    plt.show()
```

In [12]:
```python
nomor_4()
```

```
NOMOR 4 - Newton-Raphson Optimum

Kasus Konvergen: f(x) = x^4 - 4x^2, x0 = 2.5
  Konvergen di x = 1.414214 setelah 6 iterasi

Kasus Tidak Konvergen: f(x) = x^3 - 3x, x0 = 0.5
  Konvergen di x = 1.000000 setelah 5 iterasi
```



Nomor 5

Dapatkan (jika ada) nilai minimum relatif, maksimum relatif, dan titik sadel dari fungsi $f(x) = xy - x^3 - y^3$. Buatlah ilustrasinya.

$$D = f_{xx} \cdot f_{yy} - (f_{xy})^2$$

- (0, 0): D = −1 < 0 → titik sadel
- (1/3, 1/3): D = 3 > 0 dan f_xx < 0 → maksimum relatif

In [13]:
```
## Tuliskan jawaban Anda disini
def nomor_5():
    x_sym, y_sym = symbols('x y')
    f = x_sym * y_sym - x_sym**3 - y_sym**3

    fx  = diff(f, x_sym)    # df/dx
```

```python
fy  = diff(f, y_sym)    # df/dy
fxx = diff(fx, x_sym)
fyy = diff(fy, y_sym)
fxy = diff(fx, y_sym)

# Cari titik kritis
kritis = solve([fx, fy], [x_sym, y_sym])

print("NOMOR 5 - f(x,y) = xy - x^3 - y^3\n")
print(f"fx  = {fx}")
print(f"fy  = {fy}")
print(f"fxx = {fxx}")
print(f"fyy = {fyy}")
print(f"fxy = {fxy}")
print(f"Titik kritis: {kritis}")

# Filter hanya titik kritis real
kritis_real = [(xp, yp) for xp, yp in kritis if im(xp) == 0 and im(yp) == 0]

for pt in kritis_real:
    xp, yp = pt
    D = (fxx * fyy - fxy**2).subs([(x_sym, xp), (y_sym, yp)])
    fxx_val = fxx.subs([(x_sym, xp), (y_sym, yp)])
    fp_val  = f.subs([(x_sym, xp), (y_sym, yp)])

    D_val       = float(D.evalf())
    fxx_num     = float(fxx_val.evalf())

    print(f"\nTitik ({float(xp.evalf()):.4f}, {float(yp.evalf()):.4f}):")
    print(f"  D = {D_val:.4f}")
    print(f"  fxx = {fxx_num:.4f}")
    print(f"  f = {float(fp_val.evalf()):.6f}")

    if D_val > 0 and fxx_num < 0:
        print("  => MAKSIMUM RELATIF")
    elif D_val > 0 and fxx_num > 0:
        print("  => MINIMUM RELATIF")
    elif D_val < 0:
        print("  => TITIK SADEL")
    else:
        print("  => Tidak dapat ditentukan (D = 0)")
```

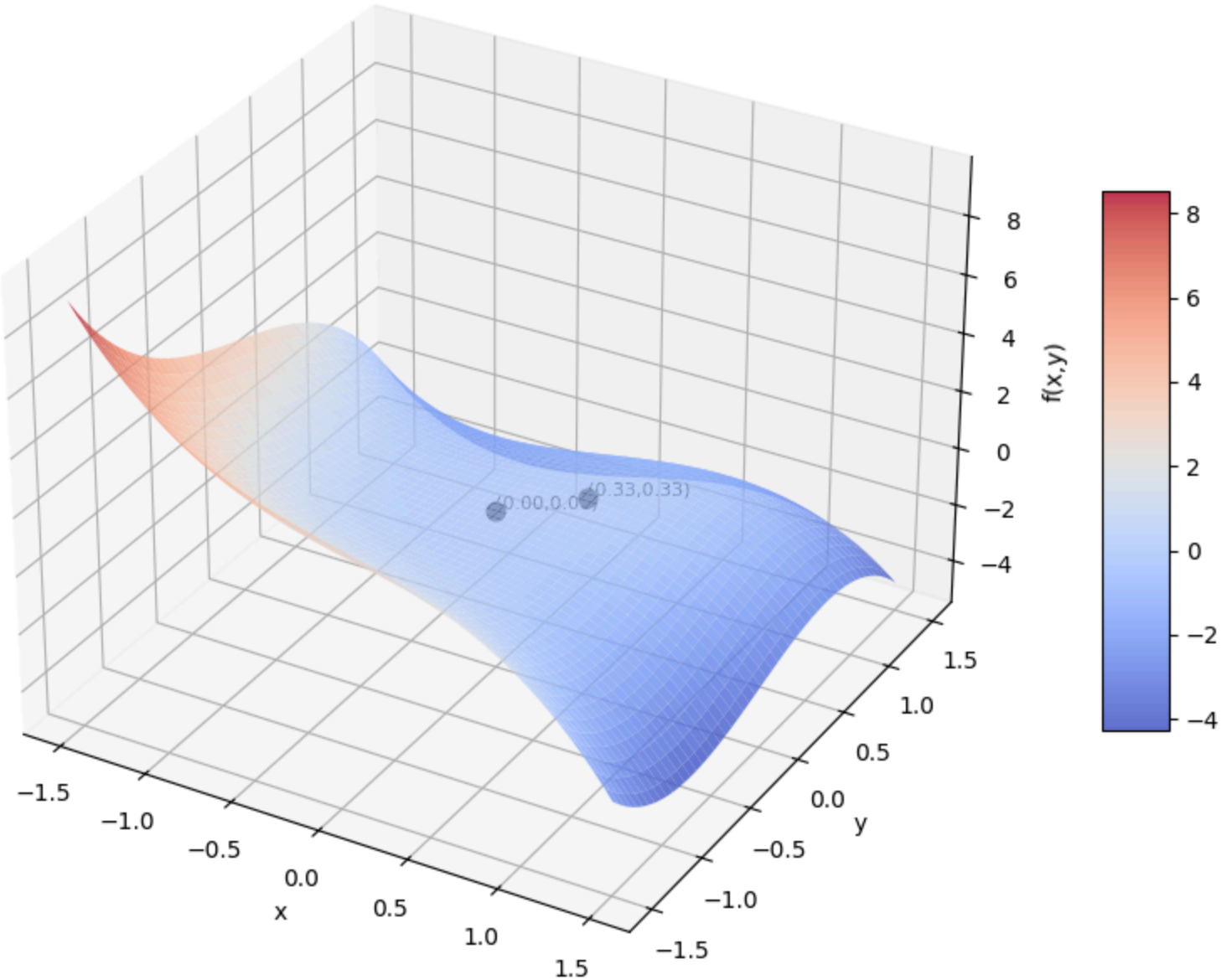
```python
    # Plot 3D
    x_vals = np.linspace(-1.5, 1.5, 200)
    y_vals = np.linspace(-1.5, 1.5, 200)
    X, Y = np.meshgrid(x_vals, y_vals)
    Z = X * Y - X**3 - Y**3

    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')
    surf = ax.plot_surface(X, Y, Z, cmap='coolwarm', alpha=0.8, edgecolor='none')

    # Tandai titik kritis
    for pt in kritis_real:
        xp, yp = float(pt[0].evalf()), float(pt[1].evalf())
        zp = xp * yp - xp**3 - yp**3
        ax.scatter(xp, yp, zp, color='black', s=60, zorder=5)
        ax.text(xp, yp, zp + 0.1, f'({xp:.2f},{yp:.2f})', fontsize=8)

    fig.colorbar(surf, shrink=0.5, aspect=8)
    ax.set_title(r'$f(x,y) = xy - x^3 - y^3$', fontsize=13)
    ax.set_xlabel('x')
    ax.set_ylabel('y')
    ax.set_zlabel('f(x,y)')
    plt.tight_layout()
    plt.show()
```

In [14]:

```python
nomor_5()
```

```
NOMOR 5 - f(x,y) = xy - x^3 - y^3

fx  = -3*x**2 + y
fy  = x - 3*y**2
fxx = -6*x
fyy = -6*y
fxy = 1
Titik kritis: [(0, 0), (1/3, 1/3), (3*(-1/6 - sqrt(3)*I/6)**2, -1/6 - sqrt(3)*I/6), (3*(-1/6 + sqrt(3)*I/6)**2, -1/6
+ sqrt(3)*I/6)]

Titik (0.0000, 0.0000):
  D = -1.0000
  fxx = 0.0000
  f = 0.000000
  => TITIK SADEL

Titik (0.3333, 0.3333):
  D = 3.0000
  fxx = -2.0000
  f = 0.037037
  => MAKSIMUM RELATIF
```

$f(x, y) = xy - x^3 - y^3$



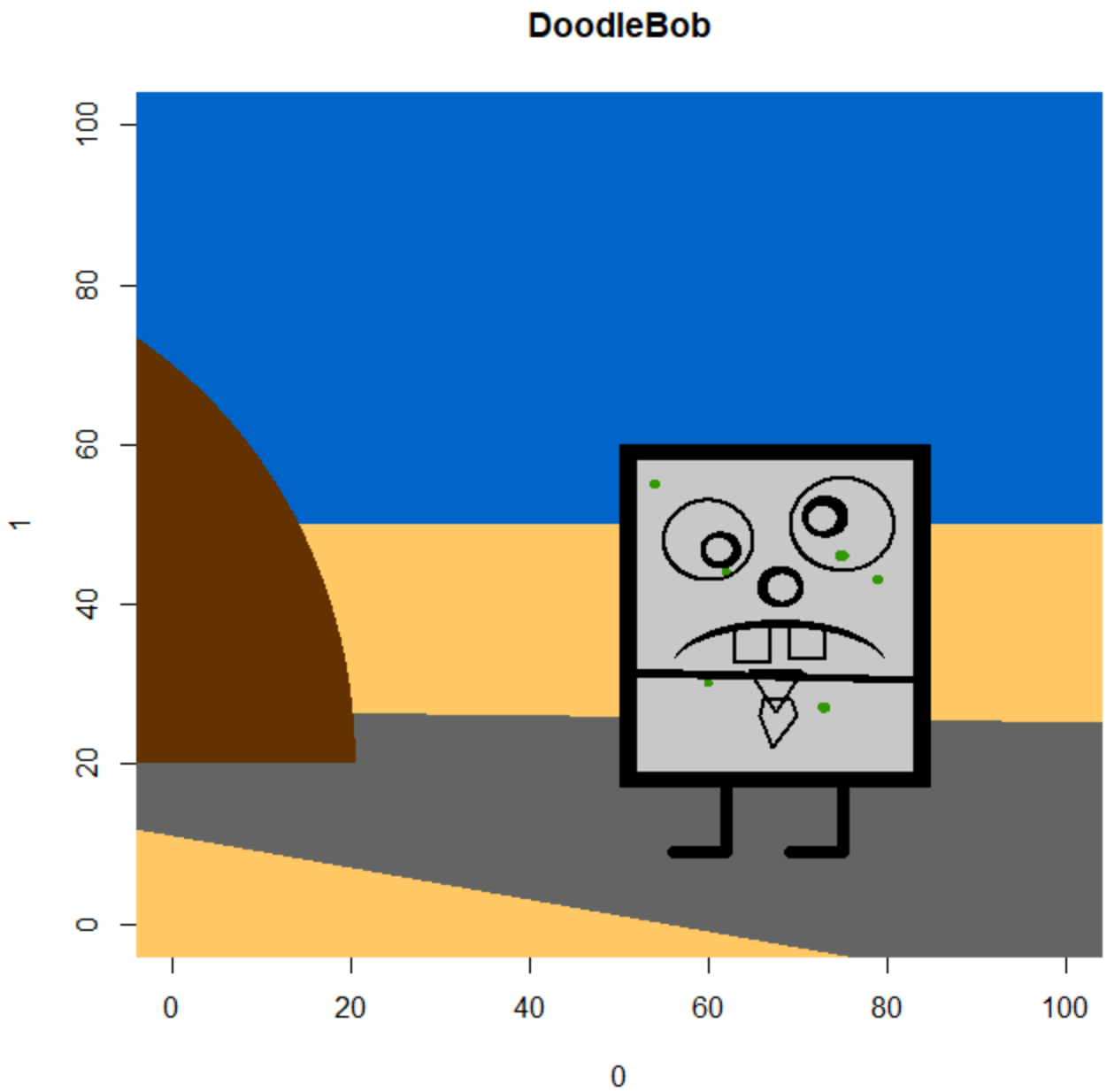
Nomor 6

Buat sebuah gambar bebas (emoji, karakter, pemandangan, dan lain-lain), dengan syarat :

- Tidak ada gambar yang sama dalam satu kelas.
- Minimum dalam gambar terdapat dua bentuk atau warna yang berbeda.

Jelaskan dan interpretasikan langkah-langkah pembentukan gambarnya. Cantumkan nama gambar yang anda buat di bagian judul plot.

Contoh Gambar:


DoodleBob
100
80
60
40
20
0
1
0
20
40
60
80
100
0

Penjelasan Gambar: Orang Sederhana

Langkah Pembentukan

1. Latar Belakang Canvas berukuran `6x6` dengan koordinat `0–100`. Background berwarna **biru langit** (`#87CEEB`) menggunakan `set_facecolor()`.

2. Matahari Dibentuk dari `plt.Circle` berwarna **kuning** di pojok kanan atas. Sinar dibuat dengan 12 garis pendek melingkar menggunakan `cos` dan `sin`.

3. Tanah `patches.Rectangle` **hijau** memanjang penuh di bagian bawah canvas sebagai permukaan tanah.

4. Badan `patches.Rectangle` **merah** diletakkan tepat di atas tanah sebagai badan orang.

5. Kepala & Wajah `plt.Circle` berwarna **kulit** sebagai kepala. Mata dari dua lingkaran hitam kecil, senyum dari arc menggunakan `np.linspace` pada sudut `200°–340°`.

6. Kaki & Tangan Kaki dari dua `patches.Rectangle` **abu-abu**. Tangan dari dua `ax.plot()` garis tebal miring berwarna kulit.

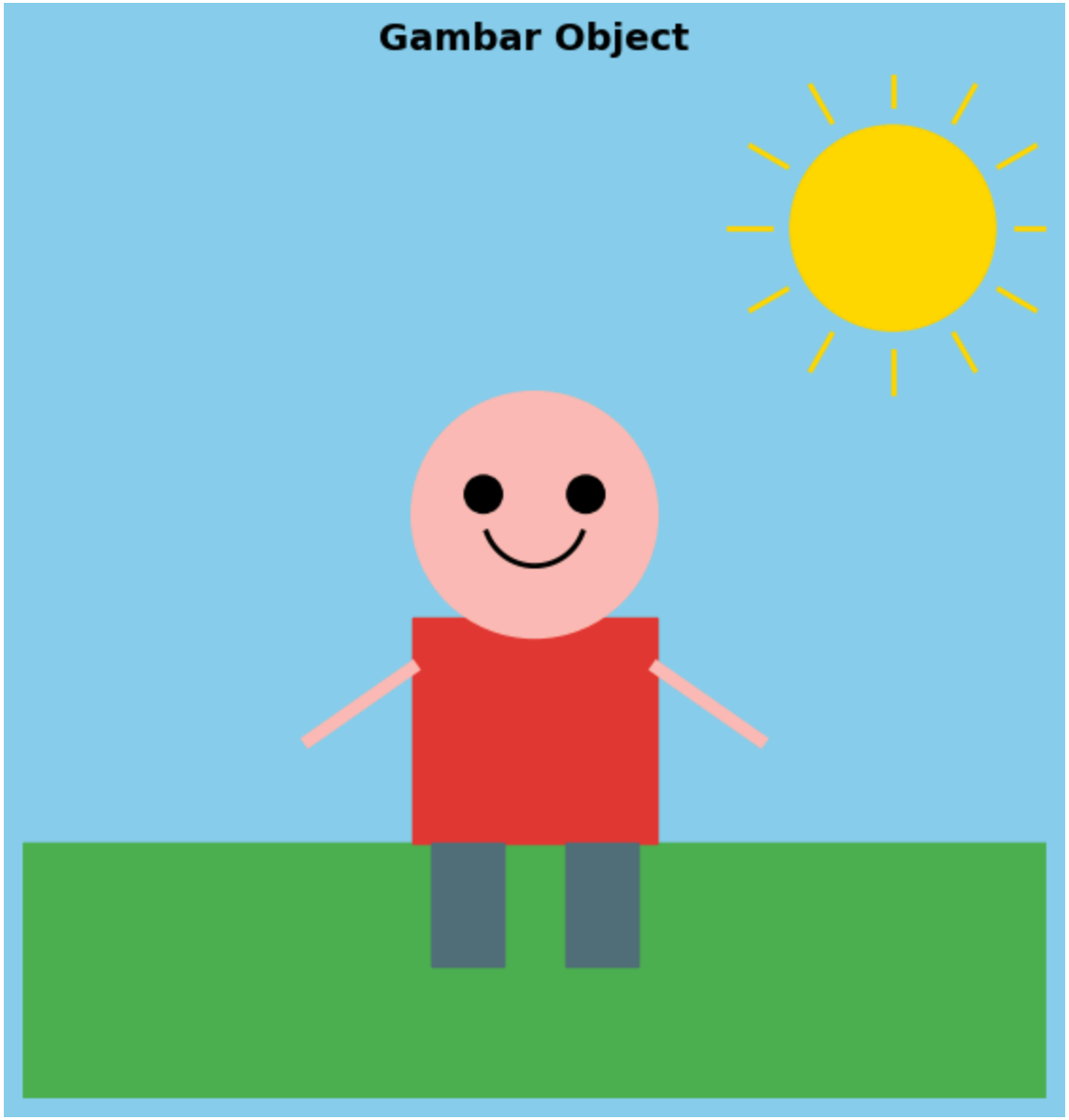
Ringkasan

Elemen	Bentuk	Warna
Matahari	Circle	Kuning
Tanah	Rectangle	Hijau
Badan	Rectangle	Merah
Kepala	Circle	Kulit
Kaki	Rectangle	Abu-abu

In [15]:

```python
## Tuliskan jawaban Anda disini
fig, ax = plt.subplots(figsize=(6, 6))
ax.set_xlim(0, 100)
ax.set_ylim(0, 100)
ax.set_aspect('equal')
ax.set_facecolor('#87CEEB')  # langit biru
fig.patch.set_facecolor('#87CEEB')

# ============================================================
# MATAHARI - lingkaran kuning di pojok kanan atas
# ============================================================
matahari = plt.Circle((85, 85), 10, color='#FFD700', zorder=3)
ax.add_patch(matahari)

# Sinar matahari (garis-garis pendek)
sudut_sinar = np.linspace(0, 2 * np.pi, 12, endpoint=False)
for sudut in sudut_sinar:
    x_start = 85 + 12 * np.cos(sudut)
    y_start = 85 + 12 * np.sin(sudut)
    x_end   = 85 + 16 * np.cos(sudut)
    y_end   = 85 + 16 * np.sin(sudut)
    ax.plot([x_start, x_end], [y_start, y_end], color='#FFD700', linewidth=2, zorder=3)

# ============================================================
# TANAH - persegi panjang hijau di bawah
# ============================================================
tanah = patches.Rectangle((0, 0), 100, 25, color='#4CAF50', zorder=2)
ax.add_patch(tanah)

# ============================================================
# BADAN ORANG - persegi panjang merah
# ============================================================
badan = patches.Rectangle((38, 25), 24, 22, color='#E53935', zorder=4)
ax.add_patch(badan)

# ============================================================
# KEPALA - lingkaran kulit
# ============================================================
kepala = plt.Circle((50, 57), 12, color='#FDBCB4', zorder=4)
ax.add_patch(kepala)
```

```python
# Mata kiri - lingkaran hitam kecil
mata_kiri = plt.Circle((45, 59), 1.8, color='black', zorder=5)
ax.add_patch(mata_kiri)

# Mata kanan - lingkaran hitam kecil
mata_kanan = plt.Circle((55, 59), 1.8, color='black', zorder=5)
ax.add_patch(mata_kanan)

# Senyum - arc
theta  = np.linspace(np.radians(200), np.radians(340), 50)
x_senyum = 50 + 5 * np.cos(theta)
y_senyum = 57 + 5 * np.sin(theta)
ax.plot(x_senyum, y_senyum, color='black', linewidth=2, zorder=5)

# ============================================================
# KAKI - dua persegi panjang abu-abu
# ============================================================
kaki_kiri  = patches.Rectangle((40, 13), 7, 12, color='#546E7A', zorder=4)
kaki_kanan = patches.Rectangle((53, 13), 7, 12, color='#546E7A', zorder=4)
ax.add_patch(kaki_kiri)
ax.add_patch(kaki_kanan)

# ============================================================
# TANGAN - dua garis miring sederhana
# ============================================================
ax.plot([38, 28], [42, 35], color='#FDBCB4', linewidth=5, zorder=4)  # tangan kiri
ax.plot([62, 72], [42, 35], color='#FDBCB4', linewidth=5, zorder=4)  # tangan kanan

ax.set_title('Gambar Object', fontsize=14, fontweight='bold', pad=10)
ax.axis('off')

plt.tight_layout()
plt.show()
```



Nomor 7

Didefinisikan T adalah variabel random nonnegatif yang menunjukkan jangka waktu sampai mendapatkan kejadian. Fungsi survival adalah probabilitas satu individu hidup (tinggal dalam suatu status) lebih lama daripada t yang didefinisikan sebagai berikut :

$$S(t) = P(T > t) = 1 - F(t)$$

Selanjutnya, fungsi hazard menunjukkan tingkat (rate) terjadinya suatu event pada saat t, diketahui belum mendapatkan event sampai saat t. Fungsi hazard didefinisikan sebagai berikut :

$$h(t) = \lim_{\Delta t \to \infty} \frac{P(t \leq T < t + \Delta t | T \geq t)}{\Delta t} = \frac{f(t)}{S(t)}$$

Kumulatif integral dari fungsi hazard disebut sebagai Hazard Kumulatif, yaitu :

$$H(t) = \int_0^t h(x)\, dx$$

Hubungan antara $S(t)$ dengan $H(t)$ adalah sebagai berikut :

$$H(t) = -\log S(t)$$

Bagian a

Diketahui suatu model survival berdistribusi Log-Logistik, yaitu:

$$f(t) = \frac{(\beta/\alpha)(t/\alpha)^{\beta-1}}{(1 + (t/\alpha)^{\beta})^2}$$

$$F(t) = \frac{1}{1 + (t/a)^{-\beta}}$$

Dengan $\alpha > 0$, $\beta > 0$, dan $t \in [0, \infty)$.

Tuliskan fungsi survival dan hazard dari distribusi Log-Logistik, kemudian buatlah tabel fungsi densitas, fungsi survival, fungsi hazard, dan fungsi Hazard Kumulatif dengan nilai $\alpha = 1$ dan variasi nilai β sebanyak 3 nilai.

- Tampilkan tabel dengan menggunakan fungsi sehingga tabel muncul secara serentak.
- Berikan judul tabel sebelum menampilkan tabel, sehingga meskipun urutan tabel tertukar, tidak akan menjadi masalah.

Bagian b

Buatlah grafik fungsi densitas, fungsi survival, fungsi hazard, dan fungsi hazard Kumulatif dengan nilai $\alpha = 1$ dan variasi nilai β yang telah digunakan di bagian (a).

- Masing-masing grafik dibuat secara terpisah (Grafik fungsi densitas sendiri, grafik fungsi survival sendiri, grafik fungsi hazard sendiri, grafik Hazard Kumulatif sendiri).
- Berikan judul dan label sumbu serta legenda yang sesuai dengan grafik.
- Grafik dibuat berwarna sehingga mudah melihat variasi nilai β yang digunakan.

$$S(t) = \frac{\alpha^\beta}{\alpha^\beta + t^\beta}$$

$$h(t) = \frac{(\beta/\alpha)(t/\alpha)^{\beta-1}}{1 + (t/\alpha)^\beta}$$

$$H(t) = \log\left(1 + \left(\frac{t}{\alpha}\right)^\beta\right)$$

- S(t) diturunkan langsung dari S(t) = 1 − F(t)
- h(t) = f(t) / S(t), dan H(t) = −log S(t)

In [16]:
```
## Tuliskan jawaban Anda disini
def survival_loglogistik(t, alpha, beta):
    return 1 / (1 + (t / alpha)**beta)

def densitas_loglogistik(t, alpha, beta):
    num = (beta / alpha) * (t / alpha)**(beta - 1)
    den = (1 + (t / alpha)**beta)**2
    return num / den

def hazard_loglogistik(t, alpha, beta):
    ft = densitas_loglogistik(t, alpha, beta)
    St = survival_loglogistik(t, alpha, beta)
    return ft / St if St > 0 else 0

def hazard_kumulatif_loglogistik(t, alpha, beta):
    St = survival_loglogistik(t, alpha, beta)
```

```python
    return -math.log(St) if St > 0 else float('inf')


def nomor_7a(alpha, beta_list, t_vals):
    print("NOMOR 7a - Tabel Distribusi Log-Logistik")
    print(f"alpha = {alpha}, beta variasi = {beta_list}")

    # Tulis fungsi survival dan hazard secara simbolik
    t_sym, a_sym, b_sym = symbols('t alpha beta', positive=True)
    F_sym = 1 / (1 + (t_sym / a_sym)**(-b_sym))
    S_sym = 1 - F_sym
    f_sym = diff(F_sym, t_sym)
    h_sym = simplify(f_sym / S_sym)
    H_sym = integrate(h_sym, t_sym)

    print(f"\nS(t) = {simplify(S_sym)}")
    print(f"h(t) = {h_sym}")

    def cetak_tabel(beta):
        print(f"\n--- Tabel untuk beta = {beta} ---")
        header = f"{'t':>6} {'f(t)':>12} {'S(t)':>12} {'h(t)':>12} {'H(t)':>12}"
        print(header)
        print("-" * len(header))

        for t in t_vals:
            ft = densitas_loglogistik(t, alpha, beta)
            St = survival_loglogistik(t, alpha, beta)
            ht = hazard_loglogistik(t, alpha, beta)
            Ht = hazard_kumulatif_loglogistik(t, alpha, beta)
            print(f"{t:>6.2f} {ft:>12.6f} {St:>12.6f} {ht:>12.6f} {Ht:>12.6f}")

    for beta in beta_list:
        cetak_tabel(beta)


def nomor_7b(alpha, beta_list, t_vals):
    print("\nNOMOR 7b - Grafik Distribusi Log-Logistik")

    colors = ['steelblue', 'tomato', 'seagreen']
    t_plot = np.linspace(0.01, max(t_vals), 500)

    judul_grafik = ['Fungsi Densitas f(t)', 'Fungsi Survival S(t)',
```

```
                    'Fungsi Hazard h(t)', 'Hazard Kumulatif H(t)']
    label_y = ['f(t)', 'S(t)', 'h(t)', 'H(t)']

    fungsi = [
        densitas_loglogistik,
        survival_loglogistik,
        hazard_loglogistik,
        hazard_kumulatif_loglogistik,
    ]

    for idx, (fn, judul, yl) in enumerate(zip(fungsi, judul_grafik, label_y)):
        fig, ax = plt.subplots(figsize=(8, 5))

        for beta, warna in zip(beta_list, colors):
            y_plot = [fn(t, alpha, beta) for t in t_plot]
            ax.plot(t_plot, y_plot, color=warna, linewidth=2, label=f'β = {beta}')

        ax.set_title(f'{judul} - Distribusi Log-Logistik (α={alpha})', fontsize=12)
        ax.set_xlabel('t')
        ax.set_ylabel(yl)
        ax.legend()
        ax.grid(True, linestyle='--', alpha=0.5)
        plt.tight_layout()
        plt.show()
```

In [17]:

```
alpha     = 1
beta_list = [0.5, 1.0, 2.0]
t_vals    = [0.25, 0.5, 1.0, 1.5, 2.0, 3.0, 4.0, 5.0]

nomor_7a(alpha, beta_list, t_vals)
nomor_7b(alpha, beta_list, t_vals)
```

NOMOR 7a - Tabel Distribusi Log-Logistik
alpha = 1, beta variasi = [0.5, 1.0, 2.0]

S(t) = alpha**beta/(alpha**beta + t**beta)
h(t) = beta*t**(beta - 1)/(alpha**beta + t**beta)

--- Tabel untuk beta = 0.5 ---

t	f(t)	S(t)	h(t)	H(t)
0.25	0.444444	0.666667	0.666667	0.405465
0.50	0.242641	0.585786	0.414214	0.534800
1.00	0.125000	0.500000	0.250000	0.693147
1.50	0.082483	0.449490	0.183503	0.799642
2.00	0.060660	0.414214	0.146447	0.881374
3.00	0.038675	0.366025	0.105662	1.005053
4.00	0.027778	0.333333	0.083333	1.098612
5.00	0.021353	0.309017	0.069098	1.174359

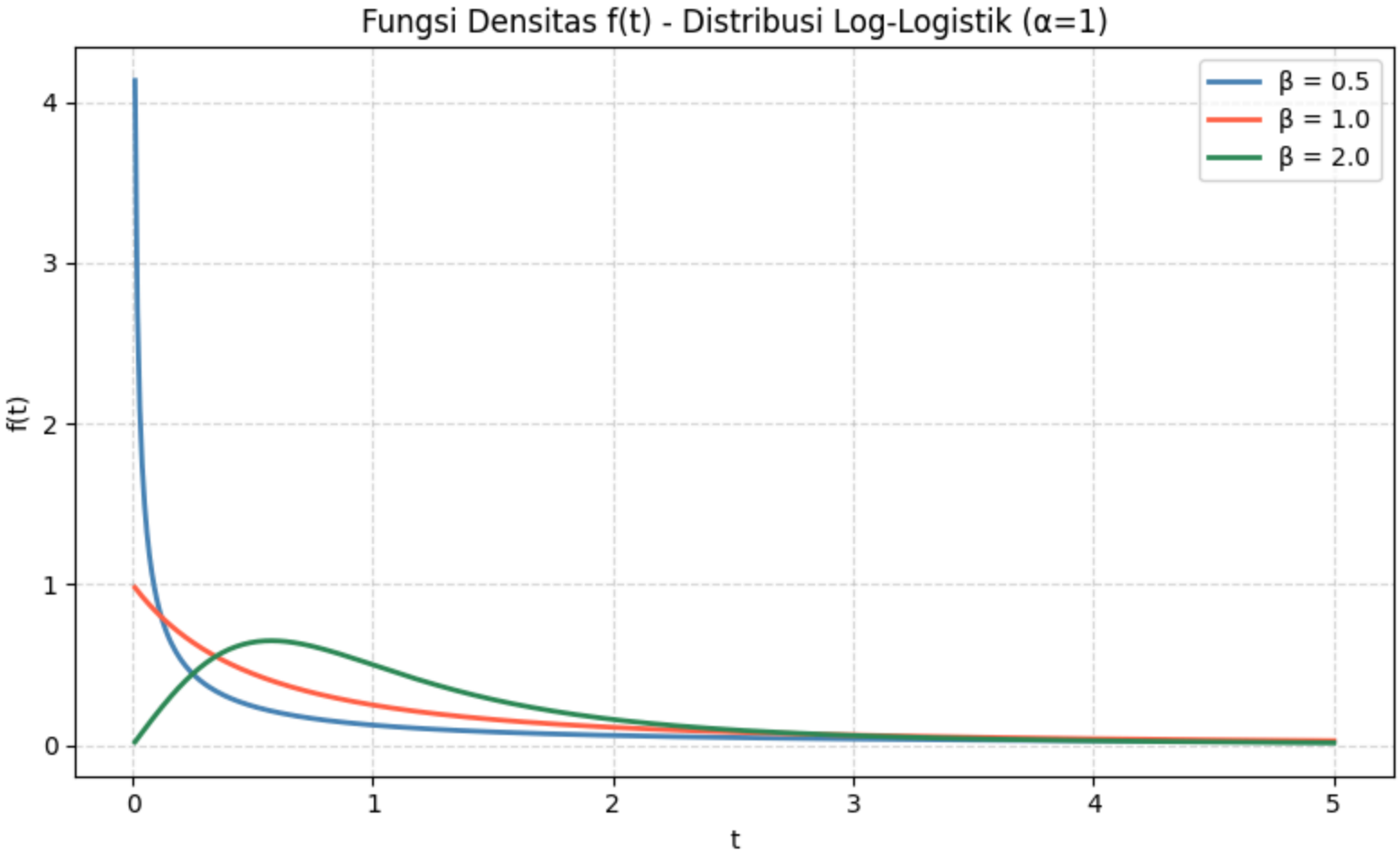
--- Tabel untuk beta = 1.0 ---

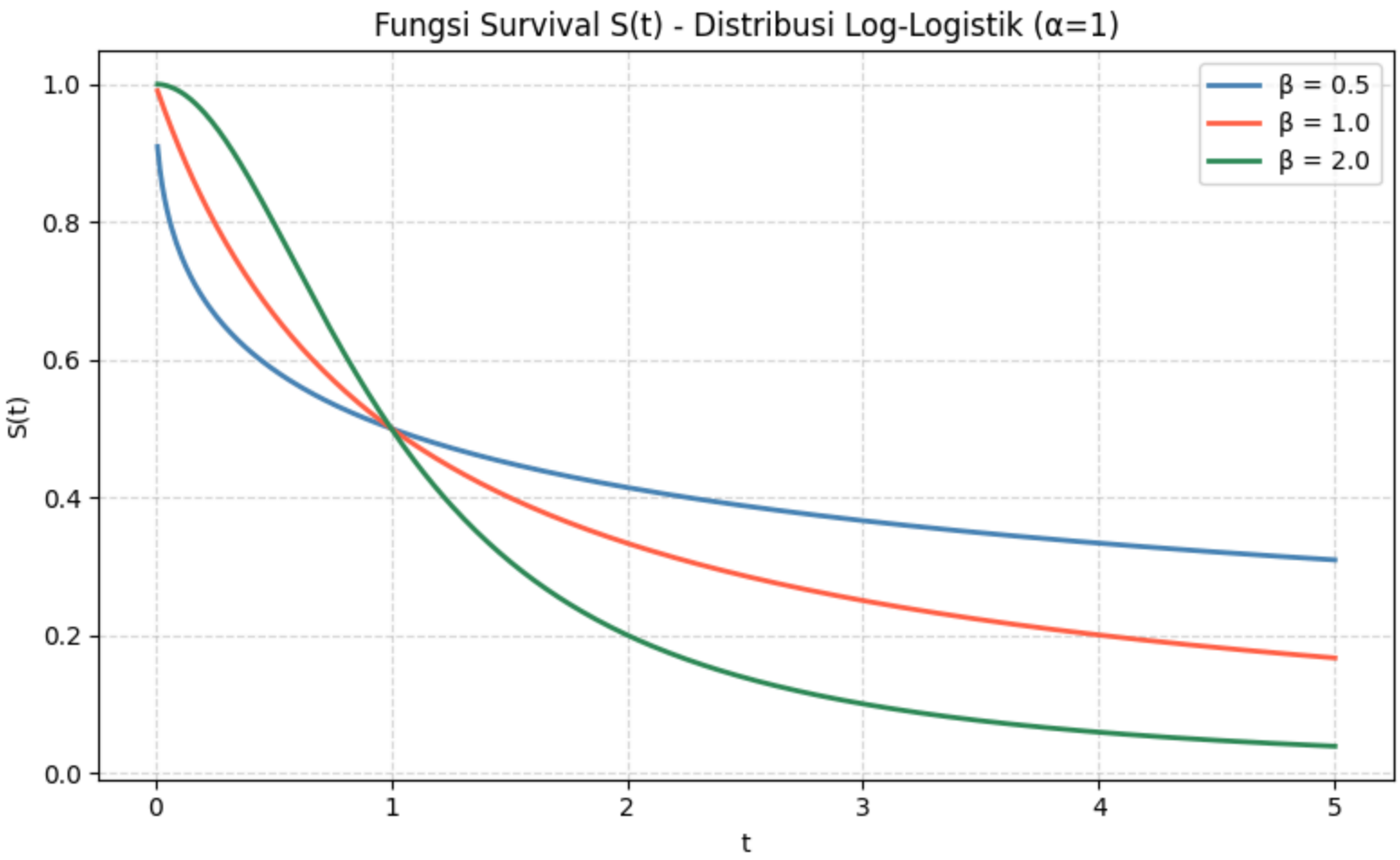
t	f(t)	S(t)	h(t)	H(t)
0.25	0.640000	0.800000	0.800000	0.223144
0.50	0.444444	0.666667	0.666667	0.405465
1.00	0.250000	0.500000	0.500000	0.693147
1.50	0.160000	0.400000	0.400000	0.916291
2.00	0.111111	0.333333	0.333333	1.098612
3.00	0.062500	0.250000	0.250000	1.386294
4.00	0.040000	0.200000	0.200000	1.609438
5.00	0.027778	0.166667	0.166667	1.791759

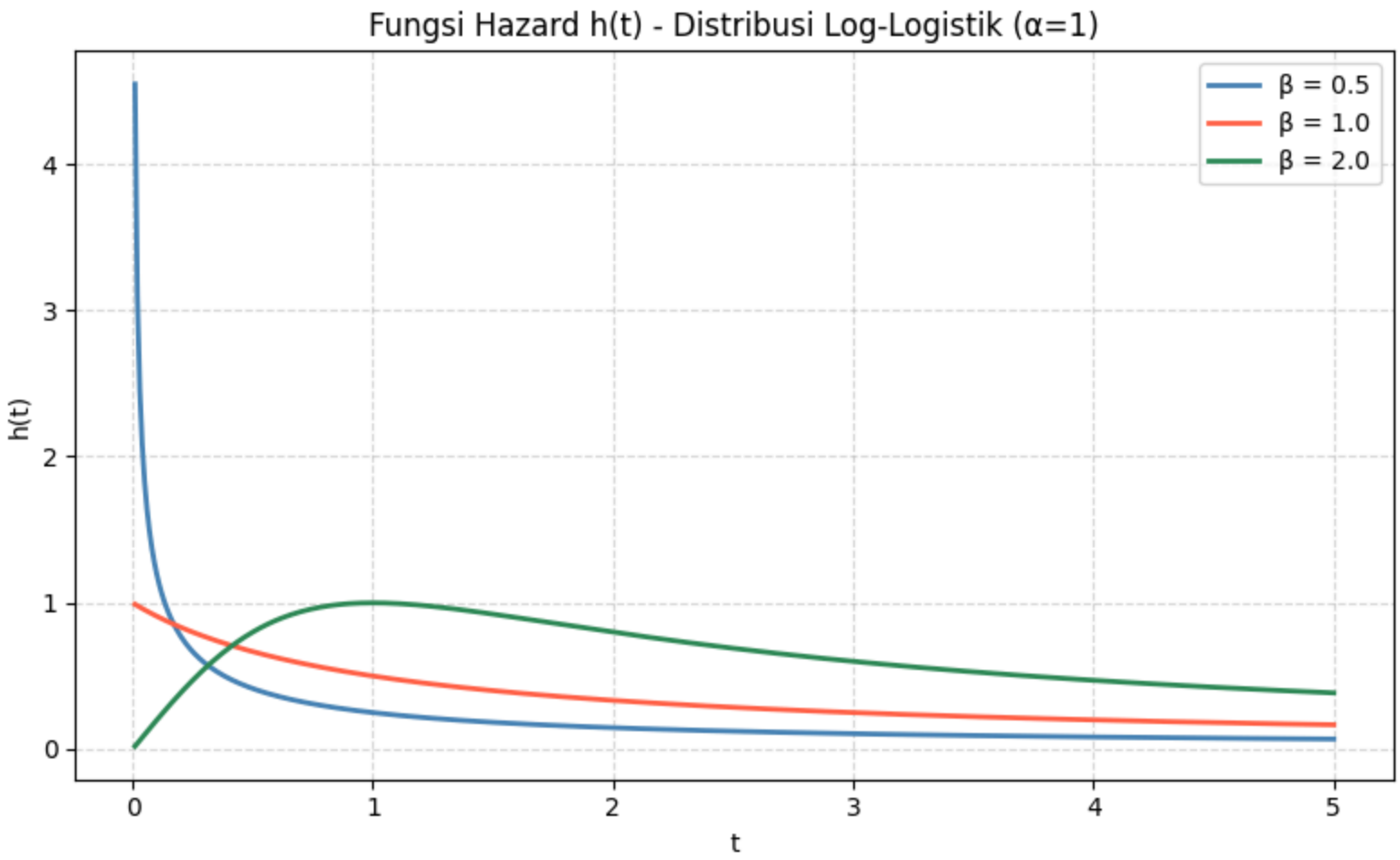
--- Tabel untuk beta = 2.0 ---

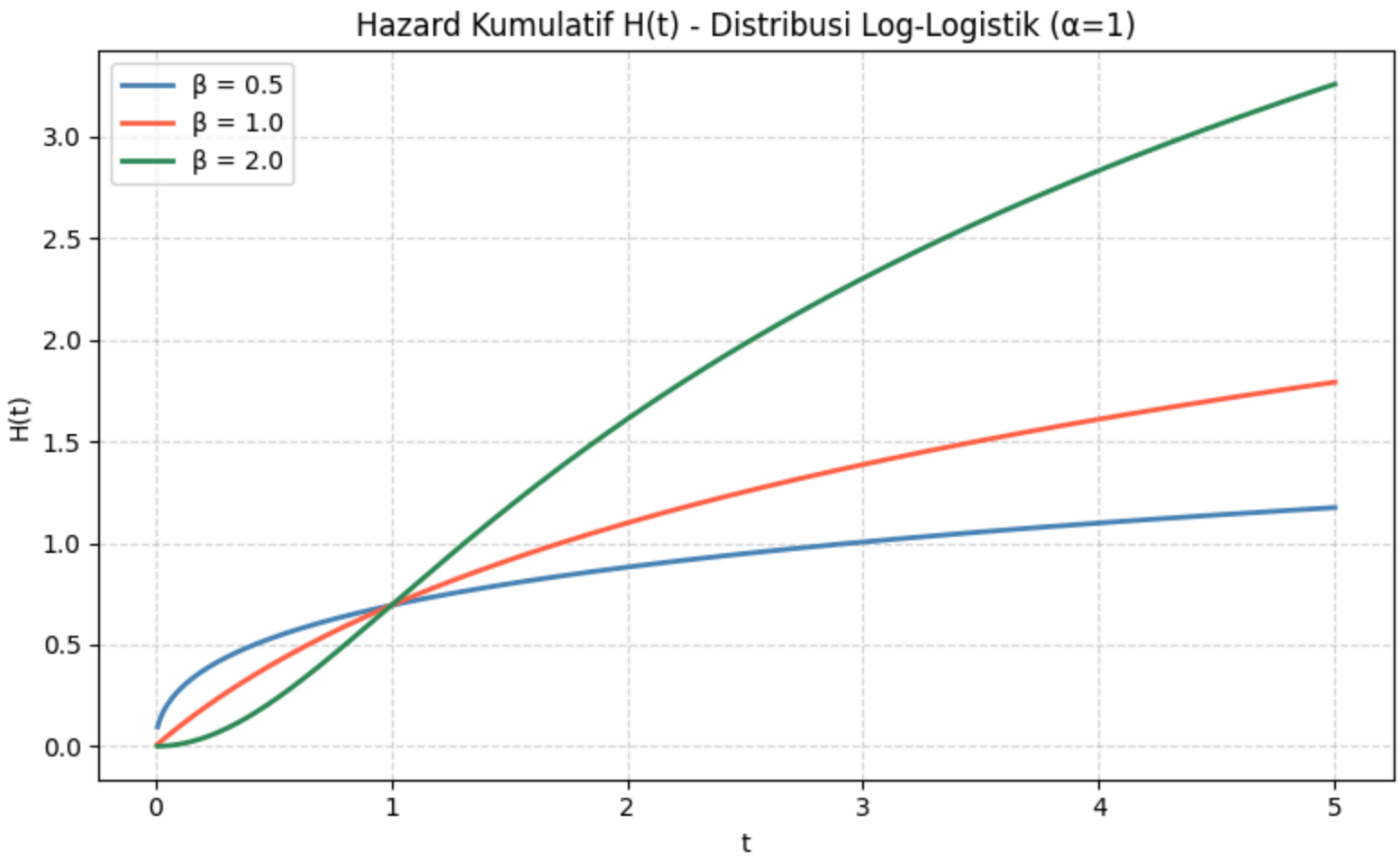
t	f(t)	S(t)	h(t)	H(t)
0.25	0.442907	0.941176	0.470588	0.060625
0.50	0.640000	0.800000	0.800000	0.223144
1.00	0.500000	0.500000	1.000000	0.693147
1.50	0.284024	0.307692	0.923077	1.178655
2.00	0.160000	0.200000	0.800000	1.609438
3.00	0.060000	0.100000	0.600000	2.302585
4.00	0.027682	0.058824	0.470588	2.833213
5.00	0.014793	0.038462	0.384615	3.258097

NOMOR 7b - Grafik Distribusi Log-Logistik




Fungsi Survival S(t) - Distribusi Log-Logistik (α=1)
β = 0.5
β = 1.0
β = 2.0
S(t)
t
1.0
0.8
0.6
0.4
0.2
0.0
0
1
2
3
4
5


Fungsi Hazard h(t) - Distribusi Log-Logistik (α=1)
β = 0.5
β = 1.0
β = 2.0
h(t)
t
0
1
2
3
4
5


Hazard Kumulatif H(t) - Distribusi Log-Logistik (α=1)
β = 0.5
β = 1.0
β = 2.0
H(t)
t
0.0
0.5
1.0
1.5
2.0
2.5
3.0
0
1
2
3
4
5